



语言分析技术实验报告

实验 2：基于 HMM 的汉语拼音输入法

姓 名：田佳铭
学 号：22920172204211
院 系：信息学院人工智能系
专 业：智能科学与技术
年 级：2017 级
指导教师：史晓东

2020 年 4 月 19 日

目录

一、实验目的.....	3
二、实验环境及工具.....	3
三、实验与附件说明.....	3
四、设计思想.....	4
1.总体设计思路.....	4
2.训练集标注.....	4
3.hmm 与输入法实现.....	5
3.1hmm.....	5
3.2 最佳匹配实现.....	5
3.3 输入法实现逻辑（无 GUI）.....	6
4.GUI 设计.....	6
5.实验结果.....	7
5.1 模型评估结果.....	7
5.2 无 GUI 界面的输出测试：.....	8
5.3GUI 界面示例.....	9
5.4 测试结果.....	10
6.现存的一些问题与可能的改进方案.....	11
五、遇到的问题与解决.....	11

一、实验目的

1. 写一个基于 HMM 的拼音输入法，可以是汉语也可以是你知道的其他语言。这里我选择汉语输入法。
2. 程序要做成 GUI 方式，概率比较大的候选要排在前面。根据输入一段文本所需要的击键次数来判断程序的好坏。
3. 采用给定的 pinyin.txt 来读取每个汉字的拼音。

二、实验环境及工具

本实验使用 python 来编写。HMM 主要使用 nltk 来实现，GUI 使用 tkinter 来实现。IDE 为 IDLE python3.7。

三、实验与附件说明

本次实验独立完成，参考了网上的一些资料，没有和同学进行内容上的讨论。实验所花时间，实验课前大约两天，主要是确定思路和数据集标注等基础工作，实验课用了一下午的时间，实验课后用了整个周六和大半个周日。

附件中文件说明：

- (1) pinyin.txt、pinyin_test、pinyin_train：实验文件夹中的文件
- (2) sim_tag.py：用简单的方式实现的数据标注程序
- (3) pinyin_tag.py：用 pypinyin 的方式实现的数据标注程序
- (4) word_seg.txt：用 sim_tag.py 实现的训练集标注文件
- (6) word_seg_new.txt：用 pinyin_tag.py 实现的训练集标注文件
- (7) word_seg_test.txt：用 sim_tag.py 实现的测试集标注文件，用于评估模型
- (8) word_seg_test_new.txt：用 pinyin_tag.py 实现的测试集标注文件，用于

评估模型

- (9) `best_path_all.py`: 根据输入拼音计算最佳匹配的程序, 是全拼方式
- (10) `best_path_sim.py`: 根据输入拼音计算最佳匹配的程序, 是简拼方式
- (11) `all_able_path_all.py`: 全拼方式的 hmm 中文输入法, 没有 GUI
- (12) `all_able_path_sim.py`: 简拼方式的 hmm 中文输入法, 没有 GUI
- (13) `all_GUI.py`: 全拼方式的 hmm 中文输入法, 有 GUI
- (14) `sim_GUI.py`: 简拼方式的 hmm 中文输入法, 有 GUI

四、设计思想

1. 总体设计思路

首先, 先要利用 `pinyin.txt` 对 `pinyin_train.txt` 进行标注, 生成标注集, 方便训练, 之后调用 `nlTK` 相关函数进行 `hmm` 模型训练, 再根据需要对输入拼音进行处理, 之后通过 `tkinter` 工具来实现简单的 GUI。

下面讲实现思路, 具体实现请根据附件说明查看相应的代码。

2. 训练集标注

除了训练集, 也对测试集做了标注, 主要用于评估 `hmm` 模型。

因为在实验课之前已经做了标注, 当时不知道可以使用 `pypinyin` 工具, 所以分了两种方式:

(1) 暴力方式。

扫描所有文本, 如果是汉字, 就在 `pinyin.txt` 中寻找对应的拼音进行标注, 寻找拼音采用暴力扫描, 即一个字以其最先出现的拼音作为标注 (如果有多个读音); 如果不是汉字, 用自身来标注自身 (数字拆开处理)。

(2) `pypinyin` 方式。

使用 `pypinyin` 生成拼音, 去除音调后进行标注。因为 `pypinyin` 会将数字整体处理, 方便起见事先消去了所有的数字。其他符号用自身标注

不管采用哪种方式, 标注格式都为 “`pinyin/汉字`”, 一个一行, 方便读取。

zhong/中
guo/国
shi/十
si/四
ge/个
bian/边
jing/境
kai/开
fang/放
cheng/城
fu/市

图 4-2-1 标注集示例

3.hmm 与输入法实现

3.1hmm

训练模式部分主要借助于 `nlk` 来实现。读入训练数据后（拼音与其对应的汉字）使用 `HiddenMarkovModelTrainer` 来构建模型，用有监督学习函数 `train_supervised` 来进行训练（就像作业 4 实现一样），并用 `evaluate` 函数，用测试集来对训练结果进行评估。

程序分为全拼和简拼两种形式，对于简拼，只需要取标签的第一个字符，重新组织训练集（和测试集）即可。

这里需要额外计算一下发射概率，但是不必要精确，这是因为 `probability` 函数无法计算一个拼音的可能性，为了应对单拼音输入而增加的。为了方便起见，使用一个汉字在全文中频率来作为发射概率（这个汉字是和拼音相关的），可以称为伪发射概率。

3.2 最佳匹配实现

在输入法之前，先对最佳匹配进行了测试，即使用 `best_path` 函数，用维特比解码的方式得到最佳路径。

3.3 输入法实现逻辑（无 GUI）

程序实现分为全拼和简拼两种形式。

在全拼的方式下，为了程序更加清晰，两个音节之间需要一个空格，而简拼则不用。

两种方式实现逻辑相似：

- （1）获得输入
- （2）如果输入是“exit”，则结束程序（否则，程序为死循环）
- （3）进行判断。如果不是合法拼音，则直接原样输出，并转向（1）（这一步的目的是为了我们能够输出标点符号之类的东西）
- （4）如果是单个音节，则根据其伪发射概率排序输出汉字与其可能性（即伪发射概率），并转向（1）
- （5）根据输入生成所有可能的路径（为了保证总是能够得到需要的结果）
- （6）使用 **probability** 计算每一个可能的路径的可能性，并进行排序。这里为了效率的话可以将可能性为 0 的路径省略。
- （7）根据排序结果输出路径和可能性，并转向（1）

4.GUI 设计

加入 GUI 后，输入法实现逻辑在原理上并没有太大改变，这里只介绍页面功能。

界面如下（简拼和全拼的界面基本上一样）：



图 4-4-1 全拼输入法的 GUI 界面

文本行分为三行：

- （1）拼音：输入栏，在这里输入拼音
- （2）结果：显示根据拼音匹配出的结果

(3) 文本：将自己需要的结果写入到文本中

按钮有四个：

(1) 退出：退出程序

(2) 生成：根据“拼音”栏在结果中生成匹配结果，一次一条

(3) 下一个：在生成后使用，生成按概率排序的下一条结果

(4) 写入：将“结果”栏的

使用方法：

(1) 在“拼音”栏中写入拼音（可以是合法拼音，也可以是其他字符）

(2) 点击“生成”按钮，经过一段时间的计算后“拼音”栏的输入会直接显示在“结果”栏（为了其他字符准备，并且有时候我们输入拼音时并不需要汉字）

(3) 如果“结果”栏中字符串不是想要的字符串，则点击“下一条”按钮，会在“结果”栏生成新的结果。这里做了控制，如果结果列表被遍历完毕，则再点击“下一条”按钮，“结果”栏不会被更新

(4) 如果“结果”栏中显示的是想要的字符串，则点击“写入”按钮，将“结果”栏中的字符添加到“文本”栏中（不是覆盖，是在原文本最后增加）。

5.实验结果

因为经过测试，使用 pypinyin 标注的效果并没有较暴力标注有明显的提高，甚至有时候效果会下降，而且因为在实验课之前已经在用暴力标注来实验了。这里主要使用的是暴力标注。

5.1 模型评估结果

全拼的 hmm 的评估结果：

| 0.9877149877149877

图 4-5-1 全拼的 hmm 的评估结果

简拼 hmm 的评估结果：

0.47174447174447176

图 4-5-2 简拼 hmm 的评估结果

可见，因为全拼的标注比简拼多，识别准确率会比简拼好很多，准确率非常高。

5.2 无 GUI 界面的输出测试：

粗略浏览训练集会发现，训练样本主要是军事方面的语句，为了能得到更好的结果，主要使用和军事有关的输入进行测试。

全拼部分测试结果：

```
请输入：zhong guo
中国 0.00980441297620933
中过 2.3889895166202056e-06
```

图 4-5-3 全拼 “zhong guo”

```
请输入：jun shi
军事 0.00010606807440580533
军士 9.278320623967065e-06
军实 1.0543546163598913e-06
军师 8.434836930879134e-07
军是 4.217418465439567e-07
军使 4.217418465439567e-07
军十 4.217418465439567e-07
军时 2.1087092327197836e-07
军世 2.1087092327197836e-07
军施 2.1087092327197836e-07
军势 2.1087092327197836e-07
```

图 4-5-4 全拼 “jun shi”

```
请输入：zhong
中 0.008434138145602291
众 0.0005015633198013946
终 0.00022596026694124261
钟 7.446417887836405e-05
忠 6.162552734761163e-05
衷 1.4550471734852745e-05
肿 1.4550471734852745e-05
仲 8.559101020501614e-07
踵 8.559101020501614e-07
家 0.0
盅 0.0
钟 0.0
鑫 0.0
肿 0.0
```

图 4-5-5 全拼 “zhong”

整体上效果不错，然后是简拼的测试：

```
请输入：zg
中国 0.00980441297620933
这个 0.004174325932887384
在国 0.0005338770652115317
整个 0.00030407941540732395
最高 0.00024719809649696234
在过 0.00018470628322484248
在该 0.0001568738295882225
中共 0.00014333937099721242
中关 0.00011467149679777
珍贵 9.825822840414195e-05
在各 9.614847619923315e-05
在公 8.349736090986034e-05
在高 6.072535338898937e-05
整合 3.9929620205002125e-05
```

图 4-5-6 简拼 “zg”


```

请输入: sj
世界 0.003278742313678561
世纪 0.00038018101536039356
事件 0.00015626808474207123
实际 8.377181889924122e-05
随即 6.661156267843819e-05
三角 5.878671540186804e-05
苏计 3.7999626132710634e-05
手机 3.634981481349778e-05
设计 3.503682157532457e-05
上解 3.2890704704200544e-05
十九 3.1151936187481264e-05
首届 3.041346749439929e-05
施建 3.0239117584288514e-05
沈觉 2.9757771738052254e-05
首家 2.8512625775999355e-05

```

图 4-5-7 简拼“sj”

```

请输入: z
在 0.01108146809124344
中 0.008434138145602291
这 0.004847018907910065
政 0.0036282029225906346
之 0.003117224591666688
总 0.0028690106620721414
作 0.0027389123265605165
资 0.0026952609113559583
者 0.002592551699109939
主 0.0022279339956365702
最 0.0018119616860401918

```

图 4-5-8 简拼“z”

效果也还可以。

5.3GUI 界面示例

GUI 示例:



图 4-5-9 输入拼音，点击“生成”按钮

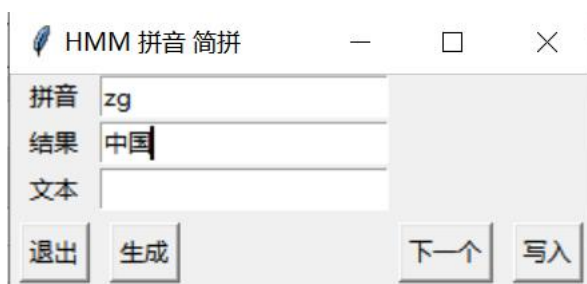


图 4-5-10 不是需要的结果，点击“下一个”按钮

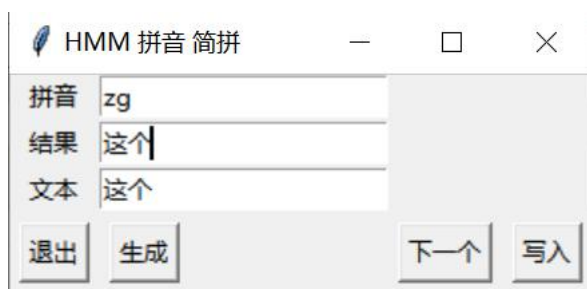


图 4-5-11 继续点击“下一个”按钮到需要的结果，点击“写入”按钮



图 4-5-12 重复操作

5.4 测试结果

将自己实现的输入法和搜狗输入法进行对比，根据点击“下一个”或者点击键盘“左”键的次数来评价好坏，公平起见，不计算标点，每次输入一或者两个拼音，对于专有名词会进行替换，而且因为 pinyin.txt 不完全，有些词语无法表示，也已经替换掉。

并且规定，如果在 100 个之内找不到就记 100

使用以下两个语句进行测试：

- (1) 它们都位于中国的东部和东北部。
- (2) 据报道，美国要求美国同意在领土展开对伊动武“北方战线”，使用美国的基地和海港。

实验结果：

句子	次数		
	搜狗输入法	全拼	简拼
(1)	10	11	58
(2)	49	95	348 (2 次没找到)

可见，相比于成熟的输入法，本次实现的输入法性能上还是比不上，特别是

简拼，很多词都很难找到。但是全拼的效果还可以，如果训练更多的话是会更好的。需要注意的是，我们实现的是静态的输入法，而实际使用的输入法会根据我们实际的使用频率进行动态的调整，多次尝试输入，输入次数会减少很多。

6.现存的一些问题与可能的改进方案

(1) 暴力标注的方案没有考虑多音字。一个很典型的例子是“厦门”，如图所示：

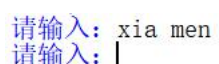


图 4-6-1 输入“xia men”

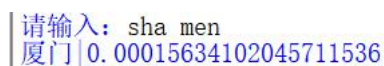


图 4-6-2 输入“sha men”

可见，因为没有考虑多音字，“sha men”对应了“厦门”。

(2) 标注集不完全，有些字错误归类了。这两点都可以通过寻找更优秀的标注集来解决。

(3) 优化做的不够，选择最优路径使用的是暴力枚举，时间复杂度较高。

(4) 训练时间长，全拼大概需要 5 分钟才能训练完毕并生成 GUI 界面，简拼大概需要 8-10 分钟。这一点可以考虑将变量保存在文件中，方便后续使用。不过本次实验保存变量总是碰到问题，所以没有这样做，这也是本次实验比较花时间的原因之一。

五、遇到的问题与解决

因为之前没怎么写过 GUI 的程序，所以做 GUI 开始不知道怎么上手，但是 python 的 GUI 设计总体上还是比较简单的，实验对 GUI 要求也不高，所以实现起来也不复杂。因为我对 python 还不是非常熟悉，很多实现都没有用 python 中一些集成的函数，而是用类似于 C 语言的方式来进行嵌套循环，函数也封装的比较少，整体上程序比较冗长。不过整体上还是没有碰到特别困难的问题。